Davante

7

BASE DE DATOS

Transformación de diagramas E/R y normalización

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Transformación al modelo relacional: Entidades fuertes 5

/ 3. Transformación al modelo relacional: Entidades débiles y de relaciones 1:1 6

3.1. Transformación de entidades débiles 6

3.2. Transformación de relaciones 1:1 6

/ 4. Transformación al modelo relacional: Relaciones 1:N y N:M 7

4.1. Transformación de relaciones 1:N 7

4.2. Transformación de relaciones N:M 8

/ 5. Transformación de generalizaciones, especializaciones y de relaciones reflexivas 8

/ 6. Normalización 9

/ 7. Dependencias funcionales: Concepto y tipos 10

7.1. Dependencia funcional completa 10

7.2. Dependencia funcional parcial 11

7.3. Dependencia funcional transitiva 11

/ 8. Primera, segunda y tercera forma normal 11

/ 9. Forma normal de Boyce-Codd 12

ÍNDICE

/ 10. Formas normales restantes: FN4, FN5 y dominio clave 14

/ 11. Desnormalización 15

/ 12. Caso práctico 1: “Diferencias entre el modelo entidad-relación y el modelo relacional” 16

/ 13. Caso práctico 2: “Paso a modelo relacional” 16

/ 14. Resumen y resolución del caso práctico de la unidad 17

/ 15. Bibliografía 18

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Entender el procedimiento de transformación del modelo entidad-relación al modelo relacional.

Aprender y dominar el concepto de normalización dentro del contexto del modelo entidad-relación.

Desarrollar habilidades en la creación de diagramas E/R mediante la práctica con diversos casos prácticos.

/ 1. Introducción y contextualización práctica

En los temas anteriores, hemos explorado cómo analizar la información de un problema dado para comenzar el diseño de una base de datos destinada a almacenar esa información. Una vez analizada esta, hemos revisado las diferentes fases del diseño, comenzando con la primera, que corresponde al modelo conceptual representado mediante diagramas E/R.

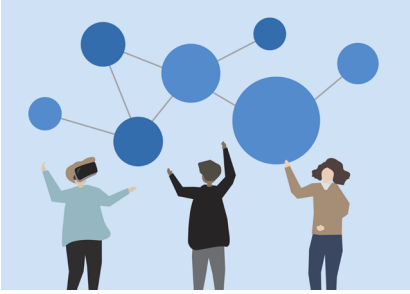

Fig.1. Practicar la transformación de diagramas es fundamental.

También hemos evaluado las ampliaciones necesarias en estos diagramas para cubrir todas las particularidades del problema a resolver. Con el primer diseño conceptual listo, procedemos al diseño lógico, más cercano al sistema de gestión de bases de datos (SGBD). Para esto, hemos estudiado los conceptos del modelo relacional.

Para construir el diseño lógico de una base de datos, se transforma el diagrama E/R al modelo relacional, aplicando reglas específicas. Estas reglas, que estudiaremos en el presente tema, se aplican a entidades, atributos y relaciones, considerando las cardinalidades del diseño conceptual.

Obtener el modelo relacional no es suficiente, además, debe estar normalizado. Esto implica verificar las **reglas de normalización y dependencia** para asegurar eficiencia y ausencia de redundancias en las bases de datos. Practicar es esencial para dominar las transformaciones necesarias, por lo que trabajaremos con casos prácticos para facilitar el aprendizaje.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio Intro. "Transformación al modelo relacional"

https://on.soundcloud.com/XnimbfiXx6I6pggi9

/ 2. Transformación al modelo relacional: Entidades fuertes

Para transformar un diagrama del modelo entidad-relación (modelo de Peter Chen) al modelo relacional (modelo de Codd), debemos seguir una serie de pasos, como la transformación de entidades, de relaciones y la normalización de todas ellas. A lo largo del tema, exploraremos en profundidad cada una de estas transformaciones.

El primer paso que debemos llevar a cabo es la **transformación al modelo relacional de las entidades fuertes**, que consiste en:

- Crear una tabla asociada a cada una de las entidades fuertes del modelo E/R. Debe contar con tantas columnas como atributos tenga la entidad en cuestión.
- Cada una de las ocurrencias de la entidad estará almacenada en una fila de la tabla.
- El atributo que sea clave de la entidad se corresponderá con la clave primaria de la tabla.

Este procedimiento puede resumirse en la siguiente tabla:

TA Elemento en el modelo E/R BLA	Elemento equivalente en el modelo relacional
Entidad	**Tabla**
Atributo	**Columna**
Identificador principal	**Clave primaria**
Identificador secundario	**Clave candidata**

Tabla 1. Transformación de entidades fuertes al modelo relacional.

En la siguiente imagen (figura 2), podemos ver un ejemplo de la entidad EMPLEADO. Esta tiene un conjunto de atributos que representan a cada ocurrencia:

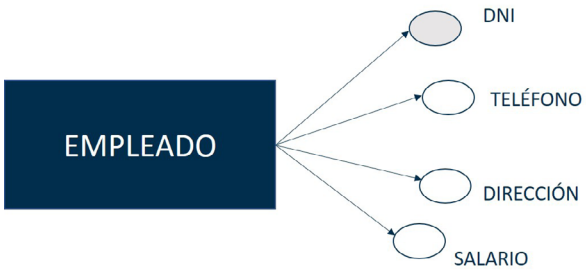

Fig.2. Entidad EMPLEADO y atributos.

Como podemos ver, la entidad está formada por varios atributos, de los cuales, el DNI es el identificador principal, ya que identifica unívocamente a cada empleado. La transformación al modelo relacional de esta entidad sería una tabla como esta:

EMPLEADOS (**DNI**, Teléfono, Dirección, Salario)

En este caso, la clave primaria sería DNI.

/ 3. Transformación al modelo relacional: Entidades débiles y de relaciones 1:1

Una vez hayamos transformado las entidades fuertes, es el turno de las entidades débiles y de las relaciones.

3.1. Transformación de entidades débiles

Las entidades débiles, tal y como hemos visto en anteriores temas, están relacionadas implícitamente con una entidad fuerte. Esta relación se hace a través de un **campo de identificación**. Transformarla a una tabla en el modelo relacional es tan simple como añadir un campo con el identificador de la entidad fuerte asociada.

En la siguiente imagen (figura 3) podemos ver un ejemplo de relación entre entidad fuerte (CUENTA BANCO) y la débil (MOVIMIENTO):

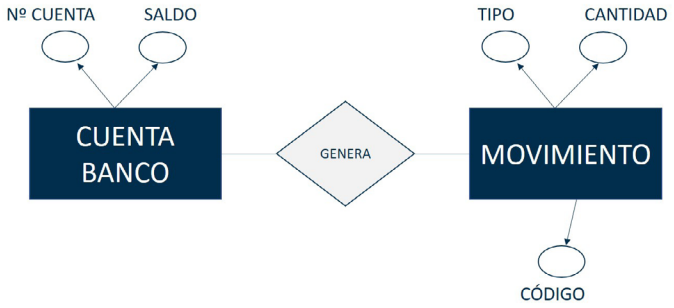

Fig.3. Ejemplo de entidad fuerte y entidad débil.

Las tablas que corresponden al modelo relacional serían las siguientes:

- CUENTA_BANCO (**N.º Cuenta**, Saldo).
- MOVIMIENTO (**N.º Cuenta**, Código, Tipo, Cantidad).

3.2. Transformación de relaciones 1:1

Existen varias formas de transformar las relaciones 1:1 al modelo relacional. De estas, ninguna supone crear una tabla, como podemos ver con el siguiente ejemplo (figura 4).

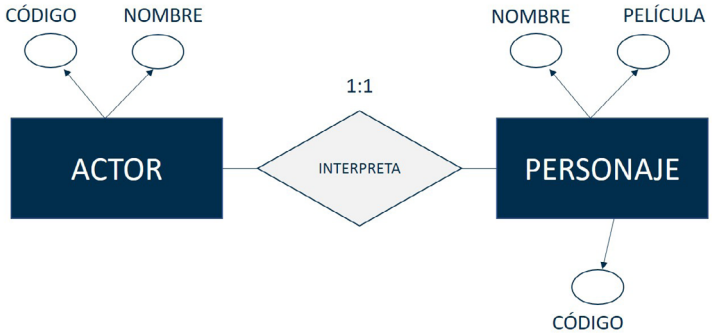

Fig.4. Ejemplo de relación 1:1.

Las diferentes formas de realizar la transformación son:

1. **Incluir la clave de PERSONAJE como clave foránea de ACTOR** (Código, Nombre, Código_Personaje).
2. **Incluir la clave de ACTOR como clave foránea de PERSONAJE** (Código, Nombre, Película, Código_Actor).
3. **Incluir la clave de PERSONAJE en la tabla de ACTOR como clave foránea y la clave de ACTOR en la tabla PERSONAJE**. En este caso, las tablas resultantes serían:
   - ACTOR (Código, Nombre, Código_Personaje).
   - PERSONAJE (Código, Nombre, Película, Código_Actor).

/ 4. Transformación al modelo relacional: Relaciones 1:N y N:M

Ahora que ya hemos transformado las entidades y las relaciones más simples, es el momento de transformar las relaciones 1:N y N:M.

4.1. Transformación de relaciones 1:N

Para realizar la transformación de este tipo de relaciones, es necesario agregar la clave de la entidad que tiene cardinalidad, 1, dentro de la entidad que tiene la cardinalidad, N, como una clave foránea. Si la relación tiene atributos, estos deben agregarse también en la entidad. Por tanto, en este caso, no es necesario generar una tabla para representar la relación entre entidades. En la siguiente imagen (figura 5) podemos ver un ejemplo de esta transformación:

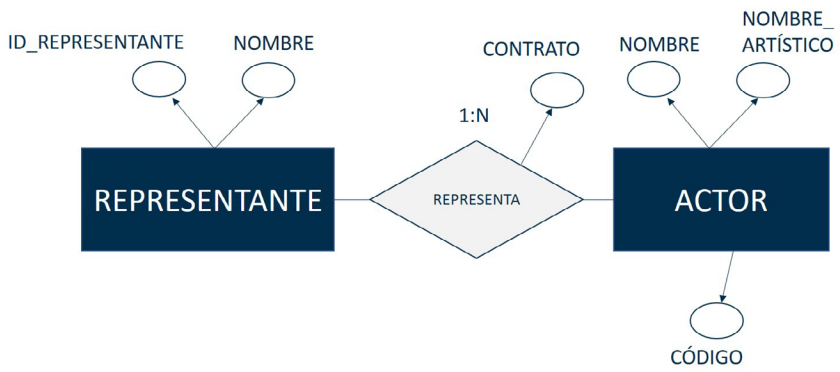

Fig.5. Ejemplo de relación 1:N.

Al transformar este ejemplo, obtendríamos las siguientes tablas:

- ACTOR (Código, Nombre artístico, Nombre, Id_Representante, Contrato).
- REPRESENTANTE (Id_Representante, Nombre).

4.2. Transformación de relaciones N:M

La transformación de las **relaciones de muchos a muchos** consiste en la creación de una tabla donde se almacenarán las claves primarias de las entidades relacionadas. Si la relación tiene atributos, estos también deben añadirse a la nueva tabla.

En el siguiente gráfico (figura 6) podemos ver un ejemplo de relación muchos a muchos que transformaremos a relacional:

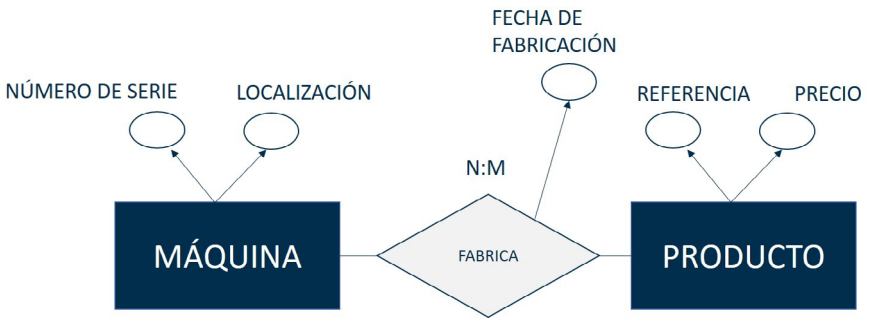

Fig.6. Ejemplo de relación N:M.

La transformación al modelo relacional daría como resultado las siguientes tablas:

- MÁQUINA (Número_serie, Localización).
- PRODUCTO (Referencia, Precio).
- FABRICA (Número_serie, Referencia, Fecha_fabricación).

Dado que la clave primaria de MÁQUINA es el Número_serie y el de PRODUCTO es la Referencia.

/ 5. Transformación de generalizaciones, especializaciones y de relaciones reflexivas

Las **generalizaciones y especializaciones pueden transformarse al modelo relacional a través de cuatro vías distintas**. Cada una de estas será más adecuada según el tipo de especialización que se esté dando en cada caso concreto (**exclusiva, inclusiva, total o parcial**).

Lo analizaremos igualmente a través de un ejemplo, partiendo del siguiente **diagrama entidad-relación**:

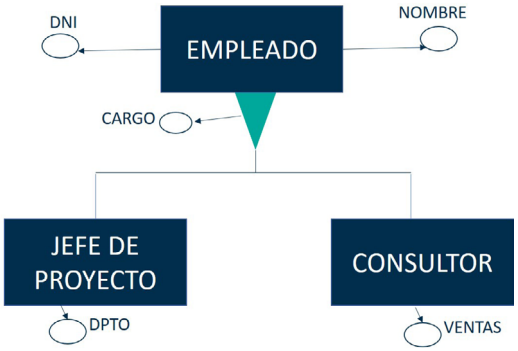

Fig.7. Ejemplo de jerarquía.

Las diferentes opciones que se pueden dar son las siguientes:

1. **Crear una tabla para la superclase y las que sean necesarias para cada subclase**, con el campo clave de la superclase. En el ejemplo anterior tendríamos:

   A. EMPLEADOS (DNI, Nombre, Cargo).

   B. JEFES DE PROYECTO (DNI, Dpto.).

   C. CONSULTORES (DNI, Ventas).

2. **Crear una tabla para cada subclase incluyendo los atributos de la superclase**:

   A. JEFES DE PROYECTO (DNI, Nombre, Cargo, Dpto.).

   B. CONSULTORES (DNI, Nombre, Cargo, Ventas).

3. **Una única tabla para la superclase, incluyendo los atributos de las subclases y un campo 'tipo'**, que indica el tipo de subclase. Esta opción es recomendable para las exclusivas:

   A. EMPLEADOS (DNI, Nombre, Cargo, Dpto., Ventas, Tipo).

4. **Una única tabla para la superclase**, pero, en lugar del tipo, se añaden varios campos. Esta opción se adapta especialmente bien a las especializaciones inclusivas:

   A. EMPLEADOS (DNI, Nombre, Cargo, Dpto., Ventas, EsJefeDeProyecto, EsConsultor).

Para las transformaciones de **relaciones reflexivas**, observaremos el siguiente vídeo para aprender el método de transformación que se utiliza en este caso:

Vídeo 1. "Transformación de relaciones reflexivas"

https://bit.ly/2LvZnQA

/ 6. Normalización

Por lo general, el proceso de diseño de una base de datos terminará con la transformación del modelo entidad-relación al modelo relacional. Este proceso debe respetar una serie de estándares conocidos como normalización.

Por lo tanto, **la normalización es un proceso que garantiza que las tablas del modelo relacional dispongan de los atributos necesarios para describir el problema de una forma coherente y organizada**.

Los objetivos de la normalización son dos, principalmente:

1. **Evitar la redundancia innecesaria de datos**: La redundancia de datos no solo conlleva un desaprovechamiento del espacio de almacenamiento, sino que puede provocar inconsistencia de datos. Reducirla, o incluso eliminarla, simplificará el mantenimiento y mejorará el rendimiento.

2. **Evitar dependencias erróneas entre elementos**: Las dependencias incorrectas entre datos pueden causar inconsistencias en los mismos, por lo que dichas dependencias deben ser coherentes para facilitar la consistencia de los datos y la integridad referencial.

El procedimiento de normalización está formado por **varias etapas secuenciales** que veremos en los próximos apartados. Estas fases son conocidas como **formas normales** (**primera, segunda, tercera, Boyce-Codd, cuarta, quinta y dominio-clave**).

Enlaces de interés...

En la siguiente página podemos visualizar varios ejemplos sobre normalización que te ayudarán a entender en mayor profundidad este concepto fundamental del modelo relacional: https://platzi.com/blog/normalizar-una-base-de-datos-y-no-morir-en-el-intento/

Para llevar a cabo este proceso de normalización, es imprescindible conocer un concepto clave: las **dependencias funcionales**. Lo veremos en el siguiente apartado.

Fig.8. La normalización garantiza el cumplimiento de una serie de parámetros o estándares.

Audio 1. "Ventajas del uso de la normalización"
https://on.soundcloud.com/SojPQM1AVGoGhCMz7

/ 7. Dependencias funcionales: Concepto y tipos

El concepto de dependencias funcionales es fundamental en el diseño y la normalización de bases de datos relacionales.

Una dependencia funcional describe cómo un conjunto de atributos en una tabla determina de manera única a otro conjunto de atributos dentro de la misma tabla.

7.1. Dependencia funcional completa

Se produce cuando un conjunto de atributos determina de manera única a otro conjunto de atributos sin que ningún subconjunto de esos primeros atributos pueda determinar al segundo conjunto.

Ejemplo: Si tenemos una tabla de empleados formada por el número de empleado y el nombre de empleado, y existe una dependencia funcional completa entre el número de empleado y su nombre, esto significa que el número de empleado determina de manera única el nombre del empleado.

{Número de empleado} →{Nombre del empleado}

7.2. Dependencia funcional parcial

Se da cuando, dentro de un conjunto de atributos que determina a otro conjunto de atributos, algunos de los atributos del primer conjunto pueden determinar al segundo conjunto sin contar con los demás atributos del primer conjunto.

Ejemplo: En una tabla de pedidos, el conjunto de número de pedido y código de producto determina la cantidad del producto. Sin embargo, el número de pedido solo puede determinar la cantidad del producto.

{Número de pedido, Código de producto} →{Cantidad de producto}

7.3. Dependencia funcional transitiva

Esta última dependencia se produce cuando un conjunto de atributos determina a otro conjunto de atributos a través de un conjunto intermedio de atributos.

Ejemplo: En una tabla de estudiantes, el número de estudiante determina el nombre del estudiante, y este, la dirección.

{Número de estudiante} →{Nombre del estudiante}
{Nombre del estudiante} →{Dirección del estudiante}
{Número de estudiante} →{Dirección del estudiante}

En el siguiente vídeo podemos ver otros ejemplos:

Vídeo 2. "Dependencia funcional, funcional completa y transitiva"
https://bit.ly/2Lwopiy

/ 8. Primera, segunda y tercera forma normal

En general, **para que una forma normal se cumpla, es necesario que la forma normal anterior también se cumpla**.

A. Primera forma normal (FN1)

La primera forma normal **prohíbe el uso de atributos con múltiples valores**. En la **tabla 2** podemos ver un ejemplo:

EMPLEADOS		
DNI	**Nombre**	**Departamento**
11111111X	Antonio	Contabilidad
22222222H	María	RR. HH. Administración

Tabla 2. Tabla que no cumple la primera forma normal.

Para que se cumpla la primera forma normal, deben desdoblarse las filas que tengan más de un valor en un atributo, por lo que el ejemplo quedaría como se muestra en la tabla 3:

EMPLEADOS		
DNI	**Nombre**	**Departamento**
11111111X	Antonio	Contabilidad
22222222H	María	RR. HH.
22222222H	María	Administración

Tabla 3. Tabla que ya cumple la primera forma normal.

B. Segunda forma normal (FN2)

Esta FN exige que **cada atributo que no sea clave debe tener una dependencia funcional completa de la clave principal**. Por ejemplo:

COMPRAS (CódProd, CódProveedor, NombreProducto, Cantidad).

En esta tabla el nombre del producto no tiene una dependencia completa **de la clave primaria** (formada por CódProd y CódProveedor), por lo que no cumple la segunda forma normal.

C. Tercera forma normal (FN3)

Prohíbe la existencia de atributos que dependan de los atributos que no son clave. En la tabla 4, el campo provincia depende del código de provincia, por lo que no está en FN3 (tercera forma normal):

EMPLEADOS			
DNI	**Nombre**	**Cód. Provincia**	**Provincia**
11111111X	Serafín	28	Madrid
22222222H	Pablo	13	Ciudad Real

Tabla 4. Tabla que no cumple la tercera forma normal.

/ 9. Forma normal de Boyce-Codd

Esta forma normal es una mejora de la tercera forma normal (FN3), por lo que para cumplirse debe hacerlo también la FN3. Fue desarrollada por Raymond F. Boyce y Edgar F. Codd para solucionar ciertos problemas que no se conseguían solucionar con la FN3.

Hemos visto que la tercera forma normal prohíbe la existencia de atributos que dependan de otros atributos que no fuesen clave primaria, es decir, las dependencias funcionales transitivas.

La forma normal Boyce-Codd exige un poco más en este aspecto, ya que para cumplirse debe darse que, en toda dependencia funcional existente, el conjunto de atributos del lado izquierdo de la dependencia funcional sea una clave candidata. Es decir, **todo atributo que no forme parte de una clave candidata debe depender de una clave candidata**.

Recuerda...

Una clave candidata es un atributo o conjunto de atributos que distingue de manera única una tupla, lo que significa que su valor no se repite en ninguna fila de la tabla. Cada tupla debe tener al menos una clave candidata, que puede ser seleccionada como clave primaria.

Esto significa que **cualquier conjunto de atributos**, ya sea simple o compuesto, **que determine otros atributos, debe ser capaz de identificar de manera única una fila en la tabla**. Una tabla en la que todos sus atributos forman parte de la clave cumplirá esta forma.

En la tabla 5, podemos ver un ejemplo en el que existe una dependencia funcional transitiva, ya que el nombre del responsable de cada empleado depende del departamento al que esté asignado ese empleado. Este, a su vez, depende del usuario en cuestión.

ORGANIZACIÓN		
Empleado	**Departamento**	**Responsable**
Antonio	RR. HH.	Ana
María	RR. HH.	Ana

Tabla 5. Tabla que no cumple Boyce-Codd.

Para representar esta tabla de modo que se cumpla Boyce-Codd, debe dividirse en dos tablas como las siguientes:

PERSONAL	
Empleado	**Departamento**
Antonio	RR. HH.
María	RR. HH.

Tabla 6. Ejemplo representativo de Boyce-Codd (1).

DEPARTAMENTO	
Departamento	**Responsable**
RR. HH.	Ana
Organización	Ana

Tabla 7. Ejemplo representativo de Boyce-Codd (1).

/ 10. Formas normales restantes: FN4, FN5 y dominio clave

En cualquier aplicación que se ponga en funcionamiento, **lo recomendable es llevar el proceso de normalización como máximo hasta la tercera forma normal (FN3) o hasta la forma normal Boyce-Codd**, ya que, por lo general, **ayudan a evitar los problemas de redundancia y de actualización**.

Estas formas normales tienen una aplicación en el mundo real que es puramente teórica.

Cuarta forma normal (FN4)

Se basa en el concepto de dependencias multivaluadas. Además, para poder cumplirse esta forma normal, la tabla no debe contener dependencias multivaluadas que no sean triviales.

EstudianteID	Curso	Afición
1	Matemáticas	Fútbol
1	Física	Fútbol
1	Matemáticas	Pintura
1	Física	Pintura

Tabla 8. Ejemplo de tabla con dependencias multivaluadas.

Para que se cumpla la cuarta forma normal, es necesario dividir esta tabla en dos tablas como las siguientes:

EstudianteID	Curso
1	Matemáticas
1	Física

Tabla 9. Ejemplo de tabla que cumple la FN4 (1).

EstudianteID	Afición
1	Fútbol
1	Pintura

Tabla 10. Ejemplo de tabla que cumple la FN4 (2).

Quinta forma normal (FN5)

Basada en las dependencias de *join* (unión). Se cumple si una tabla se puede dividir en tablas más pequeñas y las tablas resultantes pueden volver a unirse nuevamente sin perder información. En caso de que no se pierda información, se cumpliría la quinta forma normal.

Dominio-clave

Esta es la forma normal más estricta. Se enfoca en las restricciones y los dominios de valores de los atributos. Debe cumplirse FN5. Trata sobre las restricciones y los dominios de los atributos. Por ejemplo, la edad en la tabla de empleados debe estar acotada entre los 18 y los 65 años.

/ 11. Desnormalización

Consiste en incorporar redundancia controlada en un modelo de base de datos relacional normalizado. Este proceso se realiza con el objetivo principal de mejorar el rendimiento de las consultas. En términos simples, se trata de **combinar tablas que fueron separadas durante la normalización para crear estructuras menos normalizadas**, pero que permiten un **acceso y consulta de datos más eficientes**.

Las principales **ventajas al desnormalizar** son:

- **Mejora del rendimiento**: Utilizar menos pasos para combinar datos en consultas complejas hace que las respuestas sean más rápidas.
- **Simplificación de consultas**: Consolidar datos relacionados en una única tabla simplifica la escritura y ejecución de consultas complejas.
- **Simplificación de operaciones de inserción, actualización y eliminación**: Se reduce así la necesidad de mantener la integridad referencial entre múltiples tablas.
- **Eficiencia de almacenamiento y recursos**: En ciertos escenarios, una base de datos desnormalizada puede ser más eficiente en términos de almacenamiento y recursos de *hardware*.

Algunos **ejemplos comunes de desnormalización** pueden ser:

- **Tablas de resumen**: Crear tablas con totales y promedios en vez de calcular estos valores cada vez.
- **Columnas redundantes**: Añadir columnas que repiten datos para evitar unir (join) tablas al consultar.
- **Fusión de tablas**: Juntar varias tablas en una sola para evitar unirlas (join) repetidamente al consultar datos.

¡! Sabías que...

¿Las vistas materializadas (llamadas también instantáneas y que hemos visto en el tema de 'Elaboración de diagramas entidad-relación. Modelo relacional') pueden ser un ejemplo de desnormalización?

Este proceso también **conlleva algunos riesgos**, los cuales veremos en el siguiente audio.

Audio 2. "Riesgos de la desnormalización"

https://on.soundcloud.com/bJUvtAuuzQNXg2A17

En resumen, la desnormalización es un equilibrio entre optimizar el rendimiento de las consultas y mantener la integridad y eficiencia de la estructura de datos. Es una técnica que debe aplicarse cuidadosamente, considerando las necesidades específicas del sistema y las consultas más frecuentes.

/ 12. Caso práctico 1: "Diferencias entre el modelo entidad-relación y el modelo relacional"

Planteamiento: Hemos explorado diversos modelos para la modelización de datos en sistemas de bases de datos. Ahora, debemos avanzar hacia la implementación de la base de datos. Por ello, en los próximos temas, nos centraremos en la siguiente fase del diseño, conocida como diseño físico.

Nudo: Para resumir y tener una visión más clara de las características de los modelos estudiados, ¿podrías elaborar una tabla comparativa que detalle las diferencias entre el modelo relacional y el modelo E/R?

Desenlace: A continuación, se presenta un ejemplo de una tabla que podría ser adecuada para la resolución de este caso práctico.

CUESTIÓN A DIFERENCIAR	MODELO E/R	MODELO RELACIONAL
Labor	Representa la colección de objetos llamada entidades y la relación entre esas entidades.	Representa la colección de tablas y la relación entre esas tablas.
¿Qué describe?	El modelo E/R describe los datos como conjuntos de entidades, relaciones y atributos.	El modelo relacional describe los datos en una tabla compuesta por dominios, atributos y tuplas.
Relación	En el modelo E/R es más fácil entender las relaciones entre las entidades.	En el modelo relacional, las relaciones entre tablas pueden ser más difíciles de entender.
Cardinalidades	Se asignan cardinalidades a las relaciones y se visualizan en los diagramas E/R.	Las cardinalidades se aplican con restricciones de integridad y claves foráneas, sin representación visual.

Tabla 11. Comparativa entre el modelo E/R y el modelo relacional.

/ 13. Caso práctico 2: "Paso a modelo relacional"

Planteamiento: En tu trabajo, estás colaborando en el desarrollo de una base de datos y se te ha asignado la tarea de transformar los diagramas E/R en un modelo relacional.

Nudo:

Fig.9. Ejemplo de relación y entidades a transformar.

El diagrama E/R a transformar se ilustra en la figura 9, que muestra la relación entre dos entidades y sus respectivos atributos, los cuales serán transformados al modelo relacional.

Desenlace: En el ejemplo se presenta una relación M:N entre entidades, que se quiere transformar al modelo relacional. Según la teoría que hemos visto en el tema, es necesario crear una tabla intermedia que represente esta relación. Dicha tabla incluirá las claves primarias de las tablas de las entidades como claves foráneas. La combinación de estas claves foráneas constituirá la clave principal de la nueva tabla de relación.

Como resultado de la transformación al modelo relacional, obtendríamos tres tablas distintas con los siguientes atributos:

- **ALUMNOS (DNI, Nombre_AL)**.
- **CLASE (Num_Clase, Nombre_cl)**.
- **TIENE (DNI, Num_Clase)**.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema hemos analizado cómo se realiza la **transformación** de los diferentes elementos del **modelo entidad-relación al modelo relacional**.

Además, hemos visto en qué consiste el **proceso de normalización** y cuáles son las diferentes **fases** de este proceso.

Finalmente, hemos terminado analizando el concepto de desnormalización, así como sus ventajas y desventajas.

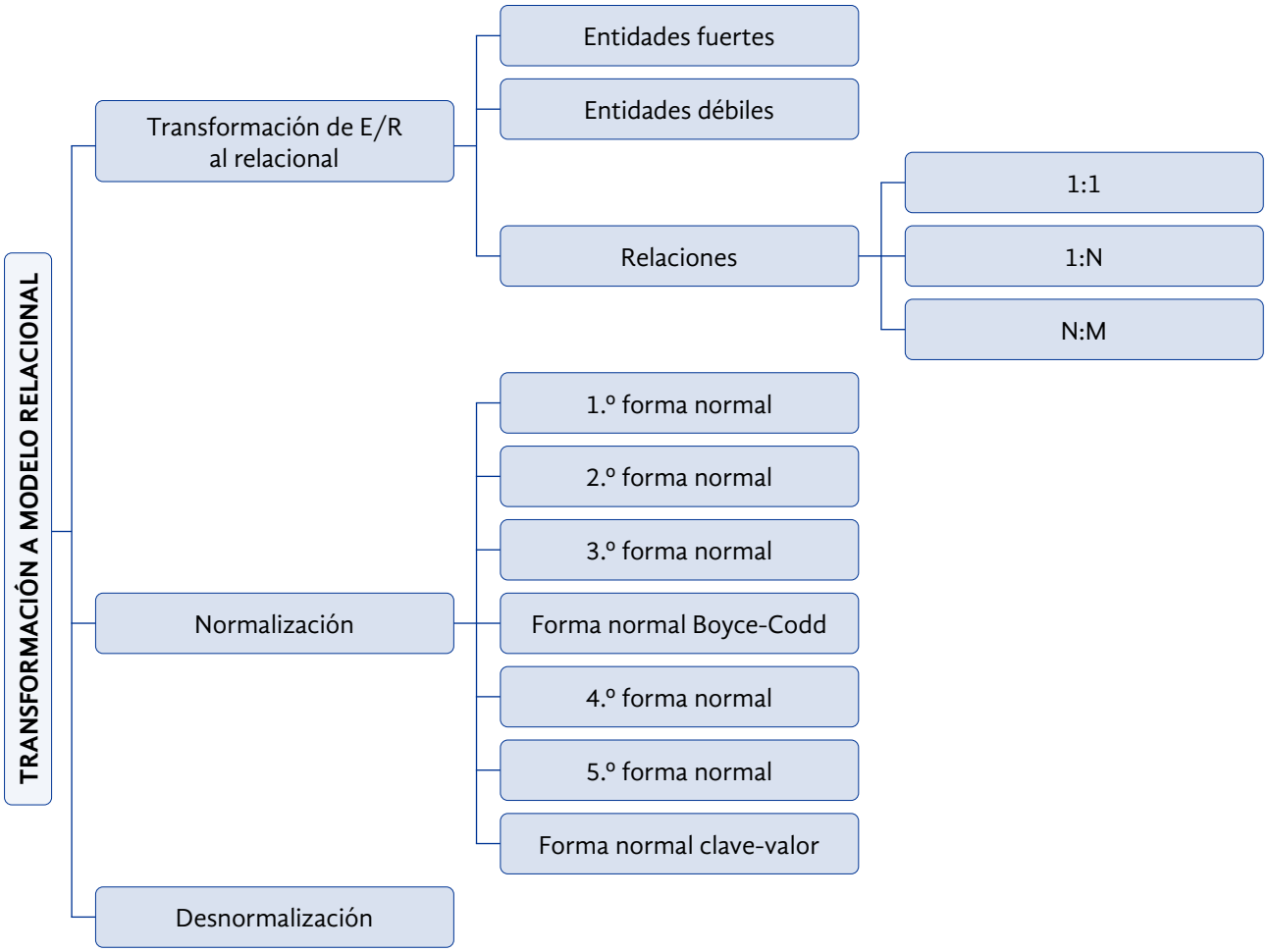

Fig.10. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Para resolver las preguntas del audio inicial del tema, debemos aplicar las reglas de transformación estudiadas durante la unidad para transformar el diagrama elaborado en el tema anterior. Podemos ver este diagrama en la figura 11.

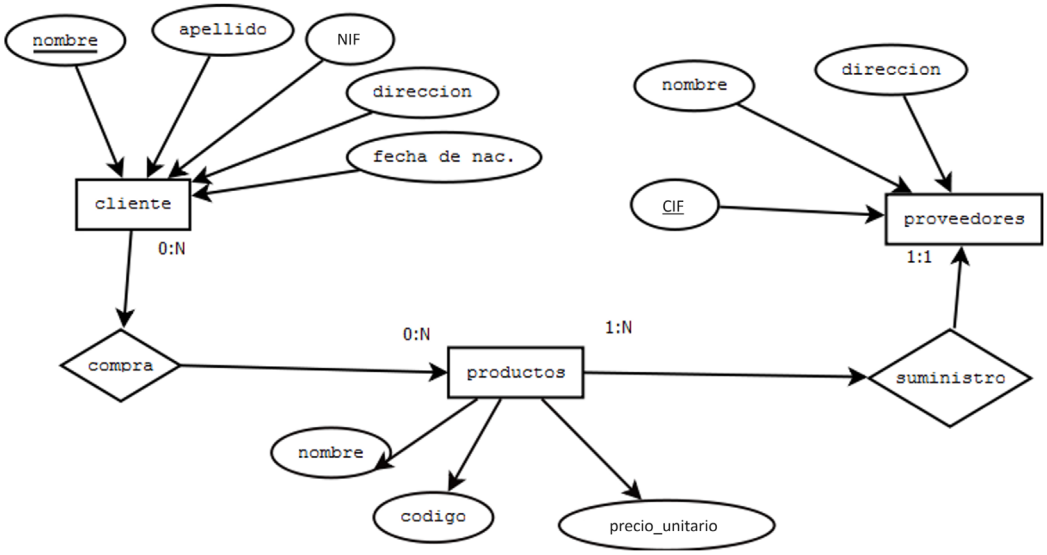

Fig.11. Ejemplo de entidades, relaciones y atributos de la tienda de informática.

El modelo relacional resultante de su transformación sería:

- CLIENTE (nombre, apellido, NIF, dirección, fecha_nacimiento).
- COMPRA (nombre, código).
- PRODUCTOS (código, nombre, precio_unitario, CIF).
- PROVEEDORES (CIF, nombre, dirección).

A partir de ahora, podremos avanzar al diseño físico y su implementación en un SGBD usando SQL, lo que estudiaremos en los próximos temas.

/ 15. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López, I., Castellano, M. J. y Ospino, J. (2011). *Bases de datos*. Garceta.

Oppel, A. (2009). *Databases A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España: Alfaomega.

Pons, O., Marín, N., Medina, J. M., Acid, S. y Vila, M. A. (2005). *Introducción a las bases de datos: el modelo relacional*. Madrid, España. Paraninfo.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.